
UNIT 15 MEMORY MANAGEMENT AND B-TREES

Structure

- 15.1 Introduction
- 15.2 Objectives
- 15.3 Memory Management
 - 15.3.1 Kinds of Data storage
 - 15.3.2 Memory Management in Python
 - 15.3.3 Memory Fragmentation
 - 15.3.4 Garbage Collection
 - 15.3.5 Reference Counting and Cycle Detection
- 15.4 Memory Hierarchies and Caching
 - 15.4.1 The Memory Hierarchy
 - 15.4.2 Caching Strategies
- 15.5 External Searching and B-Trees
 - 15.5.1 Why do we need external searching and B-Tree
 - 15.5.2 Structure of a B-Tree
 - 15.5.3 Search in a B-Tree
 - 15.5.4 Insertion in a B-Tree
 - 15.5.5 Deletion in a B-Tree
 - 15.5.6 Applications of B-Trees
- 15.6 External-Memory Sorting
 - 15.6.1 External Merge Sort
 - 15.6.2 External Sorting Techniques
 - 15.6.3 Complexity of External Sorting
- 15.7 Summary
- 15.8 Solutions/ Answers
- 15.9 Further Readings

15.1 Introduction

Efficient use of memory is one of the central concerns of designing algorithms and implementing it in any programming language. The main concern of the programmer is to design programs such that they use storage efficiently. In high-performance applications and systems handling large datasets, the way data is stored, accessed, and manipulated has a direct impact on execution speed.

The chapter introduces the concept of memory management in Python, highlighting mechanisms such as heap allocation, reference counting, and garbage collection that simplify memory handling. It then introduces the concept of memory hierarchies and caching, explaining how layered memory organization and locality of reference reduce access time and improve system performance through effective caching and replacement strategies.

This chapter further presents, B-Trees, an essential data structure for managing large sorted datasets and are commonly used in databases and file systems. B-Trees are discussed as efficient solutions for external searching reducing disk input/output operations. Searching, insertion and deletion in b-tree are explained with examples. The chapter also examines external sorting techniques, with particular emphasis on external merge sort, as practical methods for organizing large datasets stored on

15.2 Objectives

After going through this unit, you should be able to

- Understand the principles of memory management in Python and modern computer System;
 - Explain memory hierarchies and caching mechanisms
 - Explain the structure and operations of B-Trees, and
 - Apply principles of external-memory sorting, such as external merge sort.
-

15.3 Memory Management

Memory management is the process of organizing and controlling how memory is used in a computer system. It is a major concern of programming language and algorithm designers. Each algorithm designer controls the use of computer memory during program execution, including allocating, using, and freeing memory space.

15.3.1 Kinds of Data storage

Data storage during program execution can be broadly classified into primary (main memory) and secondary (external storage). Python primarily works with both through built-in mechanisms.

1. **Global or Static Storage:** In python to have global storage there are some global variables which are declared at the beginning of the program. Static variables are declared inside the function.

Example: `count = 0` # Global variable

```
def increment():
    global count
    count += 1
increment()
increment()
print(count)
```

Here, `count` remains stored for the full lifetime of the program

2. **Stack Storage:** It is a local storage that is allocated on a run time. It is used in a function or a method. Stack storage is used during execution of function calls. For example, passing parameters to function, local variables, compiler-generated temporaries (i.e. for expression evaluation) etc.

Example: `def add(a, b):` # `a` and `b` stored in stack

```
result = a + b    # result stored in stack
return result
x = 10
y = 20
print(add(x, y))
```

In this example, `a`, `b`, and `result` are created in stack memory during function execution and destroyed after return.

3. **Heap Data:** Heap is used to allocate and deallocate the memory at arbitrary times during execution. Complex storage management algorithms are needed in heap. All Python objects such as: Lists, Dictionaries, Class objects are stored in the heap.
4. **Constant Storage:** it is a storage of Literal pool. This includes string literal, Numeric Constant, Boolean values.

```
a = 100
```

```
b = 100
```

```
print(a is b) # True (same memory location)
```

Here, python reuses small immutable constants.

15.3.2 Memory Management in Python

Python uses an automatic memory-management system that handles allocation and deallocation. The main components are:

1. **Memory Heap or Python Heap:** In Python, all objects are created and stored in a common memory area known as the Python heap. Whenever a statement such as `w = Widget()` is executed—where `Widget` represents a user-defined class—a new object of that class is dynamically created and allocated space within the heap memory.

The responsibility for requesting memory from the operating system, allocating space for objects, and efficiently managing the Python heap during program execution lies entirely with the Python interpreter. This automatic memory management allows programmers to focus on program logic rather than low-level memory handling.

2. **Memory Manager:** Memory Manager controls the allocation and release of memory blocks within the heap. This can be done with the help of Memory Fragmentation techniques.
3. **Garbage Collector:** A system that automatically frees memory used by objects that are no longer needed.

The Python interpreter communicates with the operating system to request memory and manages this memory internally using optimized allocation strategies such as memory pools and arenas.

15.3.3 Memory Fragmentation

In heap-based memory allocation, the available memory is organized into contiguous blocks and managed using a linked list structure known as the free list. As memory blocks are repeatedly allocated and released, the pattern of free space continuously changes, causing the unused memory to be split into disjoint holes separated by allocated regions. This breaking up of free memory into separate holes is referred to as memory fragmentation.

Fragmentation is of two types: internal fragmentation, which occurs when allocated memory contains unused space because the reserved block is larger than the actual requirement, and external fragmentation, which occurs when free memory is scattered into many small non-contiguous blocks across the heap, making large memory allocations difficult. In figure1(a), it shows that allocated space to Process, P1 is 500 MB, But P1 utilizes only 25 MB Space and other space is wasted, cannot be used by another process. Figure 1(b) shows that if a process need 500MB, but this space is available but not as a continuous memory space.

In Python, fragmentation mainly arises due to frequent creation and deletion of objects, variable object sizes, and the dynamic resizing of data structures such as lists and dictionaries. To minimize the effects of fragmentation, Python employs efficient techniques such as object pooling, specialized small-object allocators, and automatic garbage collection.

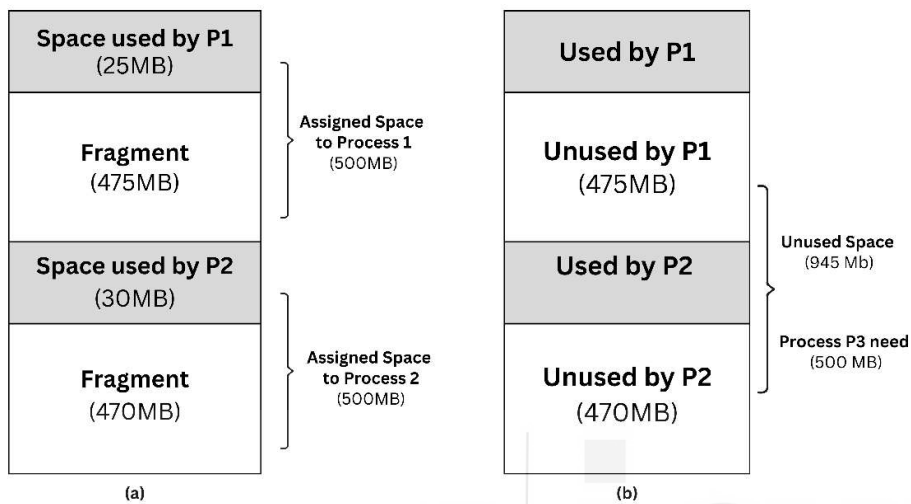


Figure 15.1: Internal and External Fragmentation

Various memory allocation strategies, such as first-fit, best-fit, next-fit, and worst-fit, are used to manage free memory blocks in the heap. The best-fit algorithm selects the smallest available block that can satisfy a request, which may lead to a higher degree of external fragmentation due to the creation of many small leftover gaps. The first-fit algorithm allocates the first suitable block it encounters, which often results in fragmentation accumulating near the beginning of the free list. The next-fit strategy continues the search from the point of the previous allocation, thereby distributing fragmentation more evenly across memory, but it can make large contiguous allocations harder to obtain over time. The worst-fit algorithm, on the other hand, allocates memory from the largest available block, with the intention of preserving sizable continuous free regions to reduce fragmentation. The selection of an appropriate allocation algorithm involves a trade-off between allocation speed, memory utilization, and the level of fragmentation produced.

15.3.4 Garbage Collection

In programming languages such as C and C++, the responsibility of explicitly releasing memory occupied by objects lies with the programmer. This task is often neglected by beginners and can lead to serious memory-related errors, such as memory leaks and dangling pointers, even in the hands of experienced developers. In contrast, the designers of Python shifted the entire responsibility of memory management to the Python interpreter. The automatic process through which unused or "stale" objects are identified, the memory occupied by them is released, and the reclaimed space is returned to the free list is known as garbage collection.

To enable automatic garbage collection, the system must first be able to identify those objects that are no longer needed. Since the Python interpreter cannot practically examine the full logical meaning of an arbitrary program, it applies a conservative reference-based rule to determine whether an object can be reclaimed. For a program to access an object, there must exist either a direct or an indirect reference to that object; such objects are classified as live objects.

A direct reference exists when an object is associated with an identifier in an active namespace, such as the global namespace or the local namespace of a currently executing function. For example, immediately after the statement, `w = Widget( )` is executed, the identifier `w` becomes a direct reference to the newly created object. Objects that have at least one direct reference are called root objects.

An indirect reference occurs when an object is referenced within the internal state of another live object. For instance, if the widget object contains a list as one of its attributes, that list is also considered live because it can be accessed indirectly through the reference `w`. This definition of live objects is recursive—any objects referenced within that list are likewise classified as live.

The Python interpreter assumes that all live objects are currently in use and must therefore be preserved in memory. Any objects that are not reachable through either direct or indirect references are considered unused and are eligible for garbage collection.

15.3.5 Reference Counting and Cycle Detection

Each Python object maintains an integer known as its reference count, which indicates the total number of references to that object currently present in the system. Python relies primarily on this reference-counting mechanism to manage object lifetimes. The reference count is increased whenever a new reference to the object is created and decreased whenever an existing reference is redirected to another object. When an object's reference count reaches zero, the object is no longer considered live, and the memory allocated to it is automatically reclaimed.

Maintaining a reference count for each object requires only constant additional space, $O(1)$, per object, and the operations of incrementing and decrementing this count also take constant time, $O(1)$, per update. The Python interpreter provides a way for a running program to inspect an object's reference count through the `sys` module, which includes the function `getrefcount( )`. This function returns an integer value representing the current number of references to the object passed as its argument.

Cycle detection addresses situations in which objects continue to reference one another but are no longer accessible from any root object, making them unreachable and therefore eligible for garbage collection. A nonzero reference count alone does not guarantee that an object is still live, because objects may exist in tightly connected groups that reference each other while remaining completely detached from the set of root objects. For example, a list may retain a positive reference count through internal links, yet become eligible for garbage collection once the external identifier that referenced it goes out of scope. To handle such scenarios, Python periodically employs advanced garbage-collection techniques, such as the mark-and-sweep algorithm, particularly when available memory becomes limited.

In the mark-and-sweep garbage collection algorithm, each object contains an associated mark bit that indicates whether it is active or inactive. When a garbage-collection cycle begins, all mark bits are initially reset. The algorithm then identifies all root objects and recursively traverses their references using a depth-first search, marking every reachable object as live; this process constitutes the mark phase. After marking is complete, the algorithm scans the entire heap and reclaims the memory occupied by all unmarked objects. During this sweep phase, the algorithm may also merge adjacent free blocks to temporarily reduce external fragmentation.

However, reference cycles (for example, self-referencing structures) can prevent cleanup. Python's garbage collector detects and frees such cyclic references.

15.4 Memory Hierarchies And Caching

With the widespread adoption of computing across all sectors of society, modern software applications are required to handle extremely large volumes of data. Such applications include online financial transaction processing, database organization and maintenance, and the analysis of customers' purchasing behavior and preferences. In many of these cases, the volume of data is so high that the overall system performance often depends more on the time required to access the data than on the raw processing speed of the CPU.

15.4.1 The Memory Hierarchy

Memory Hierarchy is the structured arrangement of various storage types that differ significantly in speed, capacity, cost, and proximity to the CPU. The primary goal of this hierarchy is to provide the illusion of a large, fast, and inexpensive memory system, even though physical limitations prevent any single memory technology from having all three characteristics simultaneously.

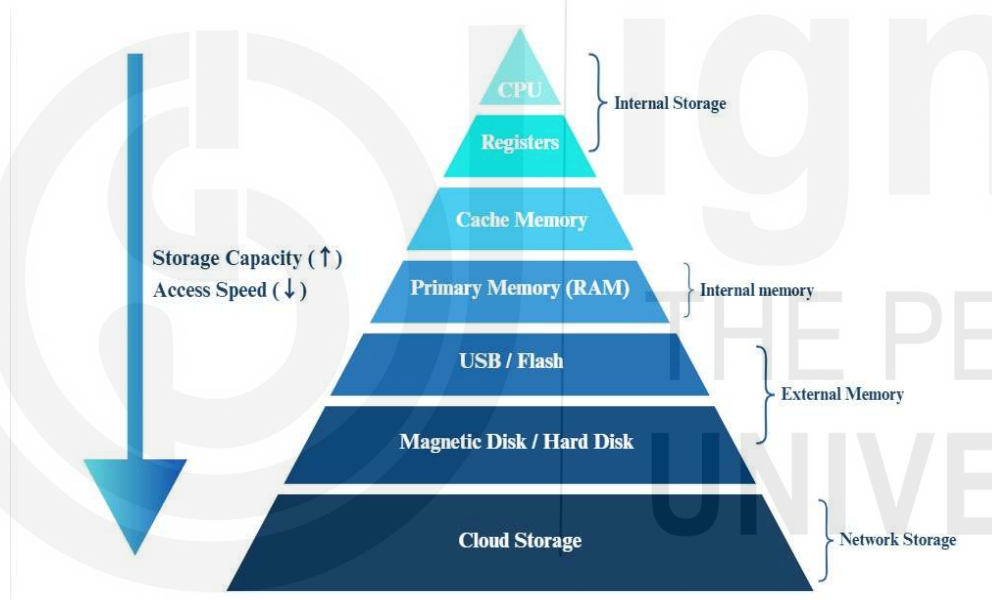


Figure15.2:Memory Hierarchy

A view of memory hierarchy is shown in Figure 15.2. It includes the following layers:

1. *Registers*: They are the nearest to the CPU, holding values needed for immediate operations. Their access to CPU is fastest, but they are small in storage space and expensive.
2. *Cache* – Small, high-speed memory located close to or within the CPU to reduce access time for frequently used data.
3. *Internal Memory (RAM)* – It has a medium-speed access to CPU, larger in capacity than register and cache memory. It is used to store actively running programs and data.
4. *External Memory*: They are the secondary storage devices with high capacity for long-term data retention but with slow access speeds. Their capacity can be increased or decreased based upon the requirement.

5. *Tertiary/Cloud Storage* – Massive, low-cost storage layers accessed infrequently or for backups.

CPU processes the data, it takes less time to fetch data through Internal memory, accessing data through external memory is expensive; disk I/O operations takes milliseconds while CPU processes data on the order of nanoseconds, one million times faster.

15.4.2 Caching Strategies

A cache stores recently or frequently accessed data to reduce latency. Whenever CPU needs the data it first checks the Cache memory. If the data is found (a cache hit), the access is very fast. If not found (a cache miss), the system must retrieve the data from lower memory levels, which is slower. Caching works on two key principles:

1. *Temporal locality*: When program elements accesses a specific memory address, it is more likely to revisit that same address again in the near future. Example, a counter variable may be referenced repeatedly—both when its value is used in expressions and when it is updated. Computer architects often summarize this behavior with the observation that a small portion of a program (around 10%) accounts for the vast majority of its execution time (about 90%).

2. *Spatial locality*: When program elements accesses one memory address, there is an increased chance that it will soon access nearby addresses as well. A common example is array processing, where programs typically read or write elements one after another, resulting in access patterns that follow a predictable, sequential flow.

To take advantage of temporal locality, it is useful to keep recently accessed web pages in a cache. Doing so allows the system to fetch these pages much more quickly if they are requested again shortly afterward. Caches are managed using hardware-controlled algorithms that decide which web pages should be loaded into the cache, when older web pages should be removed to make room for new entries, and how consistency should be maintained across multiple cache levels. To accomplish this efficiently, systems rely on various page replacement policies. Some of the policies are:

1. Least Recently Used (LRU), which evicts the page that has not been accessed for the longest time.
2. First-In, First-Out (FIFO), which removes the oldest web page entry in the cache
3. Least Frequently Used (LFU), which discards the web page that has been accessed the fewest times.
4. Random Replacement, which chooses a web page entry at random to be removed.

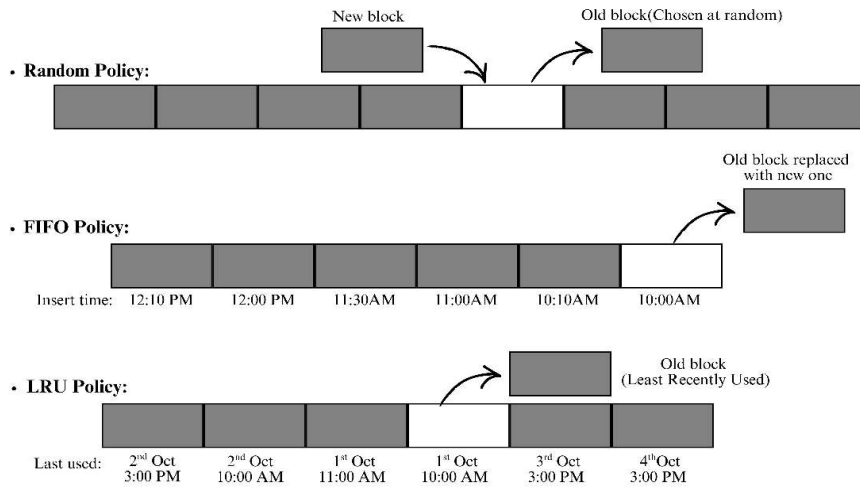


Figure15.3: Page Replacement Policies

Algorithms and data structures that access contiguous blocks of memory (arrays, B-Trees) perform better than pointer-heavy structures (linked lists) because they take advantage of caching.

Python provides software-level caching mechanisms that improve performance by avoiding repeated computation or data retrieval.

One common caching tool in Python is `functools.lru_cache`, which stores the results of expensive function calls.

Program 15.1 gives a function to compute factorial of a number in python using software caching

```
from functools import lru_cache
@lru_cache(maxsize=256)
def compute_factorial(n):
    if n <= 1:
        return 1
    return n * compute_factorial(n - 1)
print(compute_factorial(10))
```

Program 15.1: Program to compute factorial of a number showing use of software caching

This implementation ensures that repeated calls with the same argument are served from the cache, reducing computation time.

☞ Check Your Progress – 1

- 1) What is the role of the garbage collector in Python?
- 2) How does caching improve performance?
- 3) Why is memory management important?
- 4) What is fragmentation?

15.5 External Searching And B-Trees

This module presents external searching and B-tree. When data cannot fit entirely in main memory, external searching techniques are used to minimize disk accesses. B-Trees are a fundamental data structure for this purpose.

15.5.1 Why do we need external searching and B-Tree

- All the data structures discussed in this book so far favor data stored in internal memory and hence the retrieval of the data is fast. When the volume of the data is enormous, it is stored in database programs. This data is too large to fit in main memory so it is stored in disk or tapes.
- Disk access is very expensive; it takes more time to process than the processing time of CPU.
- Balanced binary search trees (AVL & Red-Black tree) have good performance if the entire data can fit in the main memory.
- These trees are not optimized for external storage and require many disk accesses, that gives poor performance for a very large data.
- The Disk accesses can be reduced, if the tree height is reduced. This can be done by increasing the number of children of each node (instead of only two child nodes). It can also be done if it stores multiple records in a block on the disk.

To achieve the above goal, we use B-trees which is a generalization of Binary Search tree. Unlike binary trees, B-Trees are designed to minimize disk reads by grouping keys into nodes that match the size of a typical disk block. This design enables fast search, insertion, and deletion, even for datasets too large to fit into main memory. Many databases and filesystems use B-Trees or their variants for indexing and data organization.

15.5.2 Structure of a B-Tree

A B-Tree of order m is a self-balancing search tree designed so that all leaf nodes appear at the same depth, ensuring that the tree always remains height-balanced. Each node contains multiple keys, and the number of children it may have lies within specific bounds. The goal is to prevent nodes from having empty subtrees and to maintain efficient access even when the dataset is too large to fit in main memory.

A B-Tree of order m is characterized by the following properties:

1. Each node can contain multiple keys stored in sorted order.
2. A B-Tree of order m permits no more than m children for any node.
3. Every internal node, except the root, maintains at least $\lceil m/2 \rceil$ children to satisfy the minimum occupancy requirement.
4. The root node contains either all keys (when the tree is a single leaf) or has at least two children in nontrivial configurations.
5. The maximum number of keys that a node may contain is $m - 1$.
6. Internal nodes other than the root must hold at least $\lceil m/2 \rceil - 1$ keys to preserve balance and density.
7. The height of the B-Tree grows or contracts only from the root, eliminating the need for structural rotations.
8. The placement of keys within each node supports efficient multiway search, reducing the number of disk or memory accesses.

Figure 15.4 depicts a B-tree of order 4, each node contains $m-1$ keys, so here maximum number of keys can be three in each node. Internal nodes hold at least 1 key to preserve balance.

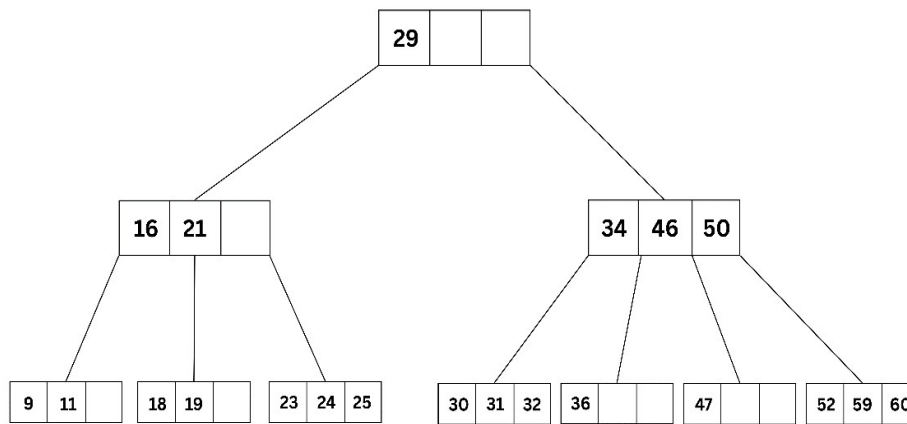


Figure 15.4:Representation of a B-tree of order 4

15.5.3 Search in a B-Tree

Searching in a B-Tree involves comparing keys to find the correct node and then traversing child pointers. Searching follows the idea as searching in a binary search tree (BST) i.e., keys on the left of a node are smaller and keys on the right are greater. However, instead of having only two children like a BST, a B-Tree node may contain multiple keys and multiple child pointers.

Insertion ensures the tree remains balanced by splitting nodes when full. The keys in each B-Tree node are stored in sorted order, keys smaller than the first key go to the leftmost child, keys between successive keys follow the corresponding internal children, and keys larger than the last key follow the rightmost child.

The search follows an alternating two-step procedure that begins at the root of the B-tree:

1. Perform a Binary search within the current node on the sorted keys stored in that node.
 - a. If a key matching the search value is found, the corresponding record is returned.
 - b. If the key is not found and the current node is a leaf, the search is unsuccessful because no further paths remain.
2. If the node is an internal node, the appropriate child pointer is selected based on the key comparisons, and the search continues recursively in that subtree.

For example, consider searching for a key with value 32 in the B-tree shown in Figure 15.4. The algorithm begins at the root node, where the comparisons lead to selecting the second (right) child. At the next level, the keys are examined again, causing the search to follow the first child pointer. This leads to a leaf node that contains the record with key value 32, and the search terminates successfully. Program 15.2 gives a simple Python method illustrating the search logic inside a B-Tree node.

```
class BTreeNode:
```

```
    def __init__(self, t, leaf=False):
```

```
        self.t = t
```

```
self.leaf = leaf
```

```
self.keys = [ ]
```

```
self.children = [ ]
```

```
    def search(self, k):
```

```
        i = 0
```

```
        # Find the first key >= k
```

```

while i < len(self.keys) and k > self.keys[i]:
    i += 1
# If the found key is equal to k, return this node
if i < len(self.keys) and self.keys[i] == k:
    return self
# If this is a leaf, the key is not found
if self.leaf:
    return None
# Otherwise search in the appropriate child
return self.children[i].search(k)

```

Program 15.2: Searching in a B-Tree

15.5.4 Insertion in a B-Tree

Insertion in a B-Tree follows a well-defined procedure that preserves all structural and balance properties of the tree. The process ensures that no node exceeds its maximum capacity and that all nodes (except the root) maintain the required minimum number of keys. A new value is inserted into the B-Tree using the following algorithm:

1. Search the B-Tree to identify the correct leaf node where the key value should be placed.
2. If the leaf node is not full, insert the new value into the node in sorted order.
3. If the leaf node is full, it means it contains $m-1$ key values, then split the node:
 - a. Insert the new key in order into the existing set of keys.
 - b. Identify the median key.
 - c. Divide the node at the median key into two separate nodes.
 - d. Move the median key to the parent node. If the parent node becomes full as a result, recursively apply the split operation to the parent.
 - e. If the root node splits, create a new root node containing the promoted median key.
 - f. The insertion finishes once no further splits are required.

Note that The insertion procedure guarantees that each node maintains a minimum occupancy of half its capacity.

Example 1: Look at the B-Tree given in Figure 15.4, Insert element 14 and element 33.

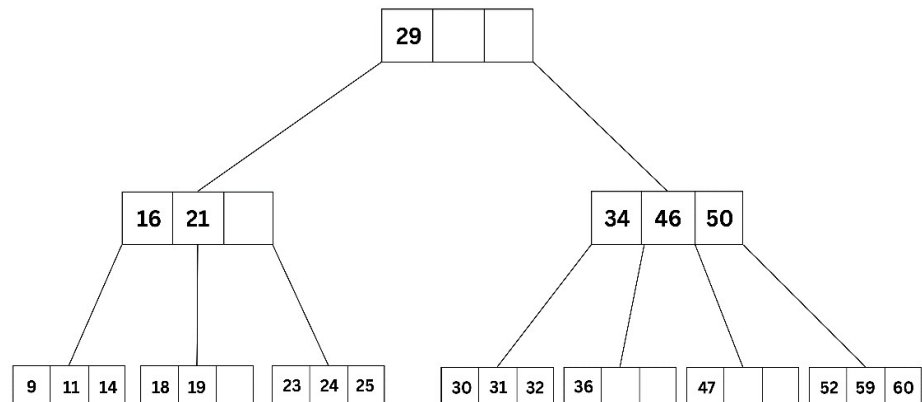


Figure 15.5: Insertion of element '14' in B-tree

In Figure 15.5, we have inserted element 14 in the tree at its appropriate position in sorted order. It is easily inserted as the leaf nodes are not full. But when we try to insert 33, it cannot be inserted directly at its position because the node full and contains three values. In this case, we need to split the node to form two different nodes. But before dividing, key values (Including the new value) are arranged in sorted order. The new order is 30,31,32 and 33. The median node value is 31, so move 31 into parent node and divide the leaf node. In this case parent node is also full and all its child nodes are already created. Split procedure will be applied here also. The new order at the parent node is 31,34,46 and 50. The median node value here is 34, so move 34 into its parent's node. The new tree is depicted in Figure 15.6.

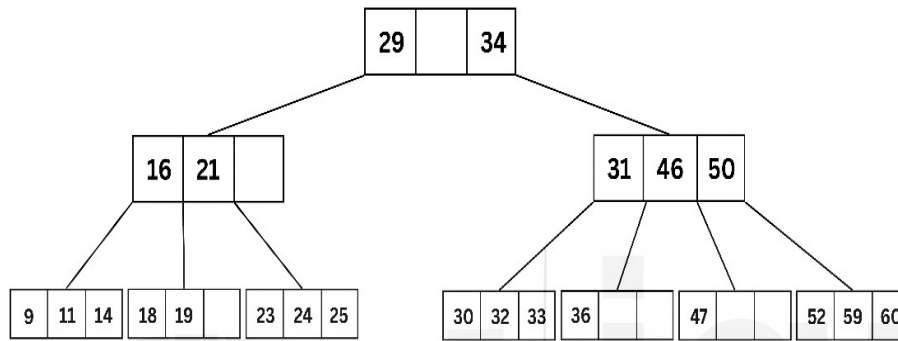


Figure 15.6: Insertion of another element '33' in a B-tree

The python code for the B-Tree Insertion is given below as Program 15.3

```

class BTreeNode:
    def __init__(self, t, leaf=False):
        self.t = t          # Minimum degree (order)
        self.leaf = leaf    # True if node is a leaf
        self.keys = []      # List of keys
        self.children = []  # List of child pointers

class BTree:
    def __init__(self, t):
        self.t = t
        self.root = BTreeNode(t, leaf=True)

    def insert(self, key):
        root = self.root
        if len(root.keys) == 2 * self.t - 1: # If root is full, tree grows in height
            new_root = BTreeNode(self.t, leaf=False)
            new_root.children.append(root)
            self._split_child(new_root, 0)
            self.root = new_root
            self._insert_non_full(new_root, key)
        else:
            self._insert_non_full(root, key)

    def _insert_non_full(self, node, key):
        i = len(node.keys) - 1

        if node.leaf:
            # Insert key into the correct position in leaf
            node.keys.append(None)
            while i >= 0 and key < node.keys[i]:
                node.keys[i + 1] = node.keys[i]
                i -= 1
            node.keys[i + 1] = key
  
```

```

else# Find the child that should receive the key
    while i >= 0 and key < node.keys[i]:
        i -= 1
        i += 1
        # If the child is full, split it
        if len(node.children[i].keys) == 2 * self.t - 1:
            self._split_child(node, i)
            if key > node.keys[i]:
                i += 1
            self._insert_non_full(node.children[i], key)
        def _split_child(self, parent, index):
            t = self.t
            full_child = parent.children[index]
            new_child = BTreeNode(t, leaf=full_child.leaf)
            parent.keys.insert(index, full_child.keys[t - 1])# Median key moves up to the parent
            parent.children.insert(index + 1, new_child)
            # Split keys
            new_child.keys = full_child.keys[t:]
            full_child.keys = full_child.keys[:t - 1]
            if not full_child.leaf:# Split children if not leaf
                new_child.children = full_child.children[t:]
                full_child.children = full_child.children[:t]
            def traverse(self, node=None):
                if node is None:
                    node = self.root
                for i in range(len(node.keys)):
                    if not node.leaf:
                        self.traverse(node.children[i])
                print(node.keys[i], end=" ")
                if not node.leaf:
                    self.traverse(node.children[-1])

```

Program 15.3: Program to Implementation of B-Tree Insertion\

To use the above code, you can call the Insert() Function and traverse () function as follows:

```

btree = BTree(t=3)

values = [10, 20, 5, 6, 12, 30, 7, 17]
for v in values:
    btree.insert(v)

print("B-Tree traversal:")
btree.traverse()

```

15.5.5 Deletion in a B-Tree

Deletion in a B-Tree is done from leaf node. As the first step, a leaf node is deleted and in the next step an internal node is deleted. The steps involved in deleting a leaf node given below:

1. Search the B-Tree to locate the leaf node that contains the key to be deleted.
2. If the leaf node contains more than the minimum required number of keys (that is, more than $\lceil m/2 \rceil - 1$ keys), simply delete the key from the node.
3. If the leaf node contains fewer than the minimum required number of keys (that is, fewer than $\lceil m/2 \rceil - 1$ keys), the node must be rebalanced by borrowing keys from its siblings or by merging nodes..
 - a. If the left sibling contains more than the minimum number of keys, transfer the largest key from the left sibling to the parent node, and move the corresponding separator key from the parent down into the

leaf node from which the deletion is being performed..

- b. Otherwise, if the right sibling contains more than the minimum number of keys, transfer the smallest key from the right sibling to the parent node, and move the corresponding separator key from the parent down into the leaf node where the deletion occurs
- c. If both the left and right siblings contain only the minimum number of keys, merge the leaf node with one of its siblings along with the separating key from the parent node to form a single leaf node. The merged node must not exceed the maximum allowed number of keys ($m - 1$). If the removal of the separator key causes the parent node to contain fewer than the minimum number of keys, the same rebalancing procedure is applied recursively to the parent. This process may propagate upward and, in some cases, result in a reduction in the height of the B-Tree.

Note: To delete a key from an internal node, replace the key with either its in-order predecessor or successor. Since this predecessor or successor always resides in a leaf node, the deletion process ultimately reduces to deleting a key from a leaf node.

Example 1: Look at the B-Tree given in Figure 15.4 of order 4, Delete element 25, then element 36 and element 46.

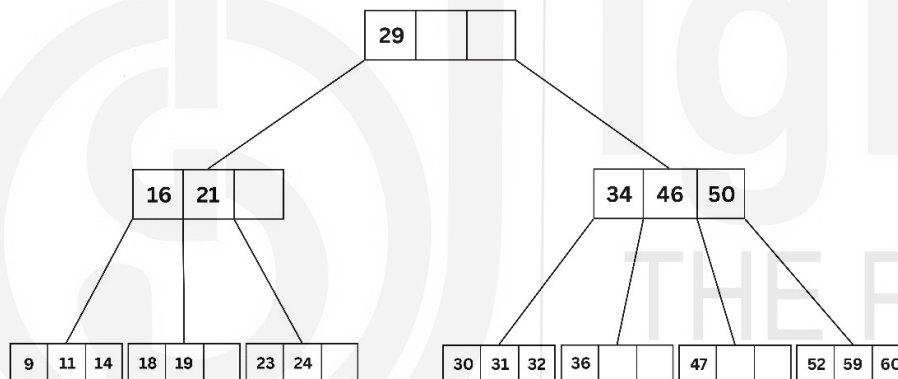


Figure 15.7: Deletion of element '25' in B-tree

When we search for element 25, it is found to be a leaf node. This leaf node contains 3 keys, which is more than the minimum required (for a typical B-Tree of order 4, minimum keys = 1 or 2 depending on convention). Therefore, simply delete it. Figure 15.7 shows the tree after this deletion.

Now consider the deletion of the key 36. The key is located in a leaf node that contains only the minimum permissible number of keys; therefore, deleting it directly would result in an underflow. To resolve this, the left and right sibling nodes are examined to determine whether they contain more than the minimum number of keys. In this case, the left sibling satisfies this condition. Consequently, the largest key (32) from the left sibling is promoted to the parent node, and the corresponding separator key (34) from the parent is moved down into the deficient leaf node. After this redistribution, the key 36 can be safely removed from the leaf node without violating the B-Tree properties. The new formed tree is shown in Figure 15.8.

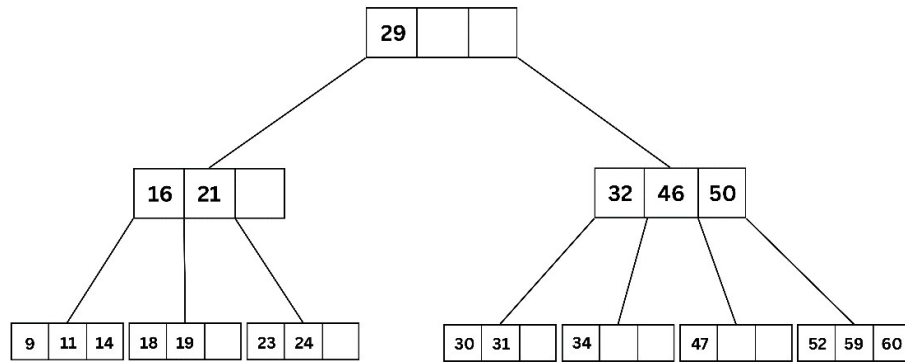


Figure 15.8: Deletion of element '36' in B-tree

Now, after tree formed in figure 15.8, delete element 46. Search for element 46. It is found to be an internal node. Here, replaces the key with either its inorder predecessor, or its inorder successor (47). 47 is located as leaf node, so move it up. Move the smallest key (52) from the right sibling up to the parent. Move the separator key (50) from the parent down to the empty leaf. Figure 15.9 shows the final tree after deleting element 46 from it.

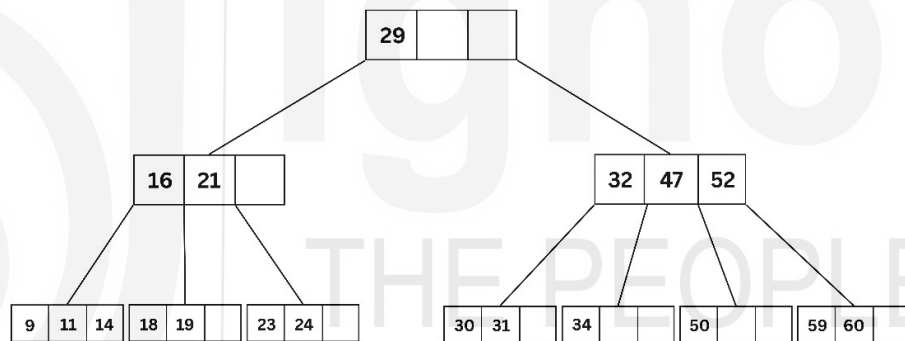


Figure 15.9: Deletion of element '46' in B-tree

15.5.5 Applications of B-Trees

B-Trees are widely used in systems that must manage large volumes of data efficiently, especially where disk access time is a critical factor. Their ability to minimize disk reads by storing many keys within a single node makes them ideal for external-memory indexing. As a result, B-Trees form the foundation of numerous database engines, such as MySQL and PostgreSQL, where they support fast insertion, deletion, and range queries. They are also integral to modern file systems including NTFS, HFS+, and ext4 which rely on them to organize directory structures and metadata. More broadly, B-Trees are employed in applications that require scalable indexing of substantial datasets, enabling quick and reliable access even when the data exceeds main memory capacity. The main applications are:

- Used in relational database indexing.
- Manage directory and metadata structures in file systems
- Support search and retrieval in large-scale data storage systems

15.6 External-Memory Sorting

In this section, we will discuss that there are many practical applications where

datasets are so large that they cannot fit entirely into the internal (primary) memory of a computer. In such cases, traditional internal sorting techniques such as Quick Sort, Bubble Sort, Heap Sort, Radix Sort, and similar algorithms cannot be applied directly. Sorting must instead be performed using external-memory sorting techniques, where data resides on secondary storage such as hard disks. In external sorting, data is transferred between disk and main memory in fixed-size units called blocks. A block consists of several records and represents the smallest unit of data that can be read from or written to disk at a time.

For a disk, there are three factors contributing to the read/write time:

- *Seek Time* – The time required for the disk's read/write head to move to the correct track where the data is located.
- *Rotational Latency* – The time taken for the disk platter to rotate so that the desired sector aligns with the read/write head.
- *Transmission Time* – The time needed to actually read or write the data once the head is positioned over the correct sector.

15.6.1 External Merge Sort

This large dataset requires external sorting techniques, such as the External Merge Sort. This method consists of two distinct phases:

Phase 1: Run Generation or Sorting Phase:

In the first phase, the large dataset residing on secondary storage is partitioned into smaller units known as runs, each of which is small enough to fit into main memory. These blocks are read into memory one at a time, sorted using an efficient internal sorting technique such as Quick Sort or Merge Sort, and then written back to disk as individual sorted runs. This process is repeated until the entire dataset has been processed, producing multiple sorted runs stored on secondary storage.

Phase 2: Merge Phase

In the second phase, the sorted runs produced during the first phase are merged to generate a single sorted output file. A multi-way merge strategy is typically employed, in which multiple runs are read concurrently into memory buffers, and the smallest elements among them are repeatedly selected and written to the output file. When the number of runs exceeds the available memory buffers, the merging process is carried out in multiple passes. This procedure continues until all runs are combined into one fully sorted dataset. By dividing the sorting process into these two phases, External Merge Sort efficiently processes very large datasets while minimizing costly disk access operations.

Algorithm To understand the working of External Merge Sort

Assume that four tapes are available on secondary storage. Two Input Tapes -Ta1, Ta2 and two output Tapes -Tb1, Tb2.

Initially the file to be sorted which is there in External memory is assumed to be on input tape, Ta1

Let, N be the total number of records (to be sorted) is on Ta1 and M be the records that can fit in the memory.

Phase 1: Run Generation

1. Divide the file on Ta1 into blocks of size M.
2. Read a block of size M from Ta1 into main memory, sort it, and store it on output tape Tb1.
3. Read the next block, sort it, and store it on Tb2.

4. Alternatively, we perform writing on Tb1 and Tb2.
5. Each sorted block produced during the run generation phase is called a run.

Phase 2: Merging of Runs

During this phase, runs from tapes Tb1 and Tb2 are merged and the results are written alternately to tapes Ta1 and Ta2.

1. Read the first run from Tb1 and the first run from Tb2 into main memory, merge them, and store the merged output on Ta1.
2. Read the second run from Tb1 and the second run from Tb2, merge them, and store the result on Ta2.
3. Read the third run from Tb1 and the third run from Tb2, merge them, and store the result on Ta1.
4. Continue this process of reading alternating runs from Tb1 and Tb2, merging them in main memory, and writing the merged output alternately to Ta1 and Ta2 until all runs have been processed.

All records which were originally on Tape Tb1 and Tb2 are shifted to tape Ta1 and Ta2. We have Tb1 and Tb2 free at this stage. In the next step retrieve data from tape Ta1 and Ta2, perform merge and sorting, store the result alternatively on Tb1 and Tb2.

5. The process is repeated until a single run is produced, resulting in the final sorted file.

15.6.2 External Sorting Techniques

Different external sorting techniques are used depending on the number of available tapes or input/output buffers. The commonly used external sorting techniques are:

1. *2-way Merge Sorting*: In this technique, two output tapes are used to merge sorted runs. Only two runs are merged at a time in each pass.
2. *K-way Merge Sorting*: In k-way merge sorting, more than two output tapes (or buffers) are available. If k output tapes are used, then k runs can be merged simultaneously. For example, when three output tapes are used, it is referred to as three-way merge sorting.
3. *Poly-phase Merge Sorting*: Polyphase merge sorting improves efficiency by unevenly distributing runs across the available tapes and eliminating the need for empty (dummy) runs, thereby reducing the total number of merge passes.

Example 1: 2-Way Merge Sorting

Given: No. of Tapes: Ta1, Ta2, Tb1, Tb2

Data/Records (initially on Ta1): 17, 3, 29, 56, 24, 18, 4, 9, 10, 6, 45, 36, 11, 43

N = 14 (total number of records)

M = 3 (memory capacity)

K = 2, incase of 2-way merge sort, which means we have two output tapes

Phase 1: Run Generation

The file is divided into blocks of size $m = 3$, and each block is sorted internally before being written alternately to the output tapes Tb1 and Tb2.

Three records are first loaded into main memory, sorted, and written to tape Tb1.

For example, the records (17, 3, 29) are sorted as (3, 17, 29) and stored on Tb1,
resulting in:
Tb1: 3,17,29

Next, another set of three records is read into main memory, sorted, and written to tape Tb2.

The records (56, 24, 18) are sorted as (18, 24, 56) and stored on Tb2, giving:
Tb2: 18,24,56

The following three records are then processed in the same manner and written to Tb1.
After sorting (4, 9, 10), the contents of Tb1 become:
Tb1: 3,17,29 | 4,9,10

Subsequently, the next group of three records (6, 45, 36) is read, sorted to (6, 36, 45),
and stored on Tb2, updating it to:
Tb2: 18,24,56 | 6,36,45

Finally, the remaining records (11, 43) are read into memory, sorted, and written to Tb1.

Tb1: 3,17,29 | 4,9,10 | 11,43

Status after Phase 1: At the completion of the run generation phase, the sorted runs are distributed as follows:

Tb1: 3,17,29 | 4,9,10 | 11,43 (three runs)

Tb2: 18,24,56 | 6,36,45 (two runs)

Ta1			
Ta2			
Tb1	3, 17, 29	4, 9, 10	11, 43
Tb2	18, 24, 56	6, 36, 45	

Phase 2: Merging of Runs

First Merge Pass (Run length = 3)

Source Tapes: Tb1(3,17,29 | 4,9,10 | 11,43) and Tb2(18,24,56 | 6,36,45)

Destination Tape: Ta1 and Ta2

In the first merge operation, the first run from Tb1 and the first run from Tb2 are read into main memory, merged in sorted order, and the resulting sequence is written to tape Ta1. The contents of Ta1 after this operation are:

Ta1: 3,17,18,24,29,56

Next, the second run from Tb1 and the second run from Tb2 are merged in a similar manner and the merged output is stored on tape Ta2. The contents of Ta2 after this merge are:

Ta2: 4,6,9,10,36,45

For third Run, no merging is required because Tb1 contains only two elements (11,43), two runs for Ta1.

Ta1: 3,17,18,24,29,56 | 11,43

Ta1	3, 17, 18, 24, 29, 56	11, 43
Ta2	4, 6, 9, 10, 36, 45	
Tb1		
Tb2		

Second Merge Pass (Run length = 6)

Source Tapes: Ta1: (3,17,18,24,29,56 | 11,43) and Ta2 (4,6,9,10,36,45)

Destination Tape: Tb1 and Tb2

In this second merge pass, the first run from Ta1 and the first run from Ta2 are read into main memory, merged in sorted order, and the resulting sequence is written to tape Tb1. After this operation, Tb1 contains:

Tb1: 3,4,6,9,10,17,18,24,29,36,45,56

Since tape Ta2 does not contain a second run, no further merging is required in this pass, and the remaining run from Ta1 is copied directly to Tb2, resulting in:

Tb2: 11,43

At this stage, each tape contains exactly one run. The final step is to merge these two remaining runs to obtain the fully sorted file.

Ta1	
Ta2	
Tb1	3, 4, 6, 9, 10, 17, 18, 24, 29, 36, 45, 56
Tb2	11, 43

Final Merge Pass (Run length= 12)

Source Tapes: Tb1: (3,4,6,9,10,17,18,24,29,36,45,56) and Tb2 (11,43)

Destination Tape: Ta1 and Ta2

In this final merge pass, Read the run from Tb1 and the run from Tb2 into main memory, merge them in sorted order, and write the resulting sequence to tape Ta1.

Ta1: 3,4,6,9,10,11,17,18,24,29,36,43,45,56

Ta1	
Ta2	
Tb1	3, 4, 6, 9, 10, 17, 18, 24, 29, 36, 45, 56
Tb2	11, 43

The tape Ta1 now contains the fully sorted values, 3,4,6,9,10,11,17,18,24,29,36,43,45,56. This example demonstrates the complete working of Two-Way External Merge Sort, including run generation, multiple merge passes, and final output formation.

The 2-way merge sort requires $\left\lceil \log_2 \frac{N}{M} \right\rceil + 1$ (initial run-construction pass) passes. For instance, if we have 14 records to be sorted with internal memory capacity of 3 records, then we will need $\left\lceil \log_2 \frac{14}{3} \right\rceil + 1 = 4$ passes, which are described below.

Example 2: K-Way Merge Sorting

In k-way merge sorting, k output tapes are used, allowing k runs to be merged simultaneously in each pass. For example, when $k = 3$, three output tapes are available for merging. Increasing the number of tapes reduces the total number of merge passes required to sort the input file. Thus, k-way merge sorting can be viewed as an extension of two-way merge sorting, designed to improve efficiency by merging multiple runs in a single pass.

The k-way merge sort requires $\left\lceil \log_k \frac{N}{M} \right\rceil + 1$ (initial run-construction pass) passes. For instance, if we have 14 records to be sorted with internal memory capacity of 3 records and 6 tapes ($k = 3$), then we will need $\left\lceil \log_3 \frac{14}{3} \right\rceil + 1 = 3$ passes, which are described below.

Given: No. of Tapes: Ta1, Ta2, Ta3, Tb1, Tb2, Tb3
Data/Records (initially on Ta1): 17, 3, 29, 56, 24, 18, 4, 9, 10, 6, 45, 36, 11, 43
N = 14 (total number of records)
M = 3 (memory capacity)
K=2, incase of 2-way merge sort, which means we have two output tapes

Pass 1: Initial run-construction pass (length of run = 3)

Ta1		
Ta2		
Ta3	This tape is not in use.	
Tb1	3, 17, 29	6, 36, 45
Tb2	18, 24, 56	11, 43
Tb3	4, 9, 10	

Pass 2: (Length of run = 9)

Ta1	3, 4, 9, 10, 17, 18, 24, 29, 56	
Ta2	6, 11, 36, 43, 45	
Ta3	This tape is not in use.	
Tb1		
Tb2		
Tb3		

Pass 3: (Length of run = 14 because of last pass and there are only 14 records to be sorted)

Ta1		
Ta2		
Ta3	This tape is not in use.	
Tb1	3, 4, 6, 9, 10, 11, 17, 18, 24, 29, 36, 43, 45, 56	
Tb2		
Tb3		

As per the concept of k-way merge, we need 2k tapes. However, in some cases when the records to be sorted are less in number, it may be possible to sort the records with reduced number of tapes. For example, using 3-way merge (which requires 6 tapes), sorting of 14 records (with internal memory capacity of 3 records) can be done with 5 tapes.

In our example, it can be seen that tape Ta3 is available for sorting but it is not used in the sorting process, due to less number of records.

Example 3 (Disk Based example)

Consider a file containing 5500 records that must be sorted using a computer memory with an internal memory capable of sorting at most 850 records. The input list is maintained on disk and has a block length of 250 records, An additional disk is available for use as a scratch disk, while the original input disk is read-only.

During the run generation phase, blocks of data are read from the input disk into primary memory, sorted internally, and written back to the scratch disk as sorted

runs. Because each block contains 250 records, the input file consists of 22 blocks.

Number of blocks = $5500/250 = 22$ blocks.

Given that internal memory can accommodate up to 850 records, only three blocks (750 records) can be processed at a time. $850 \div 250 = 3.4 \Rightarrow 3$ blocks per run. Consequently, the algorithm produces seven initial sorted runs ($5500/850 = 6.47$ approx. 7 runs), six of which contain three blocks each, while the final run contains the remaining single block. Each run is read from the input disk, sorted in primary memory and then written back to scratch disk.

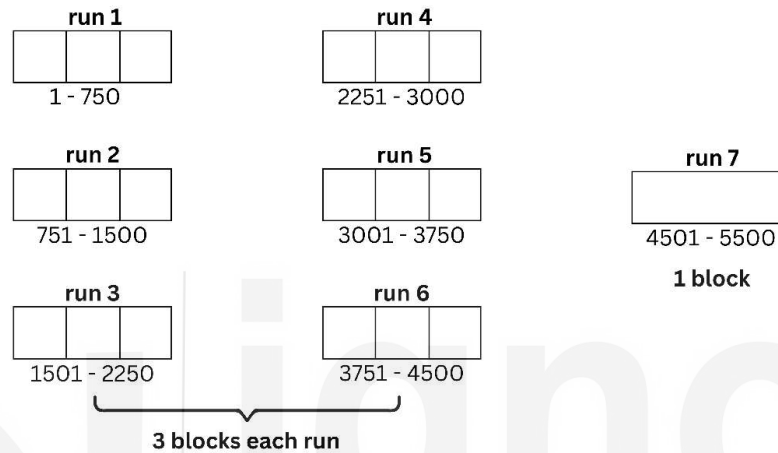


Figure 15.10: Blocks of data in each run-in run generation phase

In Merge phase, the sorted runs stored on the scratch disk are combined to produce the final sorted output. One block of memory is reserved as an output buffer (i.e., 250 records), and the remaining memory ($850 - 250 = 600$ records) allows for two input buffers ($600/250 = 2$), enabling a two-way merge. The runs are merged in successive passes, reducing the number of runs in each pass until only one fully sorted file remains. In this example, three merge passes are required to combine all seven runs into a single sorted sequence.

In pass 1: Run 1 merged with Run 2 gives Run 12, Run 3 merged with Run 4 gives Run 34, Run 5 merged with Run 6 gives Run 56, Run 7 remains.

In pass 2: Run 12 merged with Run 34 gives Run1234, Run 56 merged with Run 7 gives Run567.

In pass 3: Run1234 is merged with Run567

15.6.3 Complexity of External Sorting

The performance of external sorting is primarily determined by disk input/output operations, as disk access is significantly slower than CPU computation. Let N denote the total number of records, B the number of records per disk block, and M the capacity of main memory. The total number of disk blocks required to store the data is given by N/B .

During the run generation phase, each block is read once and written once, resulting in $2(N/B)$ disk I/O operations. In the merge phase, the entire dataset is read and written in each merge pass. If k merge passes are required, the I/O cost is $2k(N/B)$. Hence, the total I/O complexity of external sorting is: $2(1+k)N/B$.

Since the number of merge passes grows logarithmically with the number of runs, the overall complexity is:

$$O\left(\frac{N}{B} \log_{\frac{M}{B}} \frac{N}{B}\right)$$

In the example, the total number of records is $N = 5500$, the block size is $B = 250$ records, and the main memory can hold $M = 850$ records. Hence, the total number of disk blocks is: $N/B = 5500/250 = 22$ blocks. $2 * 22 = 44$ disk I/O operations will be performed. 3 merge phases are required since 3 merge passes are required, the merge phase incurs $2 \times 3 \times 22 = 132$ disk I/O operations. Therefore, the total disk I/O cost of the external sort is:

$44 + 132 = 176$ block I/O operations

This example demonstrates that the performance of external sorting is primarily governed by disk input/output operations rather than CPU processing time.

☛ Check Your Progress – 2

- 1) Draw a B-Tree with elements 40,10,20,80 of order 3.
- 2) In the above Tree Insert a) elements 70 and b) element 30.
- 3) In a B-Tree of order m , what is the maximum number of children that a node can have?
- 4) Name the most commonly used external sorting technique. _____.
- 5) What is the role of main memory in external sorting?
- 6) What type of disk access is preferred in external sorting?

15.7 Summary

This chapter highlights the use of memory management and data structures in determining overall computer system performance. Efficient utilization of memory resources was shown to be as important as algorithmic efficiency, particularly when handling large volumes of data. Python automatically manages memory using memory management techniques, such as heap allocation, reference counting, and garbage collection. These techniques were discussed as effective tools for simplifying memory handling while ensuring safe and efficient execution.

The chapter further explored memory hierarchies and caching, emphasizing how layered memory organization and locality of reference significantly reduce data access time. Various caching and replacement strategies were introduced to demonstrate how frequently accessed data can be retained in faster memory to enhance performance.

As data sizes grow beyond the capacity of main memory, the focus shifts to disk-friendly algorithms and data structures like B-Trees and external sorting. B-Trees were presented as an efficient solution for external searching, minimizing disk input-output through balanced structure and high branching factors. Additionally, external sorting techniques, particularly external merge sort, were analyzed as practical approaches for sorting large datasets that reside on secondary storage.

Together, the concepts presented in this chapter provide a strong foundation for designing scalable, high-performance systems capable of managing real-world, large-scale data efficiently.

15.8 Solutions / Answers

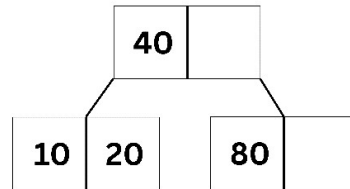
Check Your Progress 1

1. The garbage collector in Python automatically identifies and frees memory occupied by objects that are no longer in use, particularly handling reference cycles that cannot be reclaimed by reference counting alone.
2. Caching reduces average memory access time by storing frequently used data

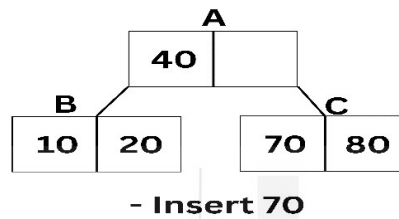
closer to the CPU.

3. It ensures efficient memory use and prevents memory leaks.
4. Fragmentation is wastage of memory due to unused gaps between blocks.

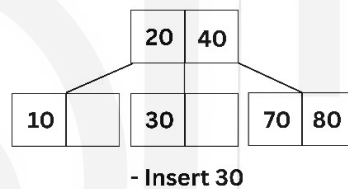
CheckYour Progress2



- 1.
2. (a)



- 2 (b)



3. m
4. Merge Sort
5. Main memory is used to hold manageable portions of the data for sorting and merging before writing the results back to disk.
6. Sequential disk access is preferred in external sorting because it is faster and more efficient than random access.

15.9 Further Readings

ReferenceBooks

1. Horowitz, E., Sahni, S., and Mehta, D. *Fundamentals of Data Structures in C++*. Universities Press (India) Pvt. Ltd., Hyderabad, 2008.
2. Goodrich, M. T., Tamassia, R., and Goldwasser, M. H. *Data Structures and Algorithms in Python*. John Wiley & Sons, 2016.
3. Horowitz and Sahni, *Fundamentals of Computer Algorithms*, Computer Science Press, 2008